DOC-SMCTA-GOV-000 · VERSIÓN 2.1.0

Arquitectura Transaccional Determinista para Mercados Secundarios P2P de Derechos de Consumo con Custodia Segregada

Estudio de Ingeniería de Software, Verificación Formal de Concurrencia y Auditoría Estructural sobre el Monorepo SMCTA v2

Proyecto / Sistema	**División de Ingeniería de Software y Sistemas Distribuidos — Documentación Oficial de Blindaje Técnico**
Fase / Área Normativa	**06 — Gobernanza de Ingeniería y Blindaje Técnico Oficial**
Autoría y Gobernanza	**Equipo de Ingeniería y Arquitectura — Proyecto SMCTA (Roles 1, 2 y 3)**
Fecha de Emisión	**Noviembre 2026**

Resumen

El diseño de mercados digitales para la reventa de entradas, reservas hoteleras y cupones gastronómicos enfrenta tres patologías recurrentes: el riesgo de insolvencia financiera cuando la plataforma actúa como contraparte directa (*Market Maker*), la reclasificación regulatoria del activo como instrumento financiero especulativo y la condición de carrera de doble gasto (*double-spending*) bajo ráfagas de concurrencia. Esta investigación presenta la arquitectura, fundamentación matemática y verificación empírica de **SMCTA v2** (*Sistema de Mercado Comportamental y Trading de Activos*), una plataforma *White-Label* multi-inquilino construida como monorepo modular en TypeScript estricto sobre Node.js 20, Fastify 5, React 19 y PostgreSQL 15. El sistema sustituye la recompra directa por un libro de órdenes *Peer-to-Peer* (P2P) acotado por bandas dinámicas de precios (*Price Collar*), respaldado al 100% de su valor nominal mediante una cuenta de custodia (*Escrow*) inmutable y un protocolo de redención física basado en tokens dinámicos `HMAC-SHA256` con versionado monótono anti-captura. La auditoría estática y dinámica sobre las **593 funciones productivas** (`6.846` líneas netas de código) demuestra **cero infracciones de complejidad ciclomática** ($\mathrm{CC} \leq 5$, $\mathrm{CCE} = 0$), un Índice de Mantenibilidad medio de $\mathrm{MI} = 87{,}08$ (factor de fricción operativa $\mathrm{F}(\mathrm{MI}) = 1{,}00\times$), distancias a la Secuencia Principal de Robert C. Martin estrictamente acotadas ($\mathrm{D} \in [0{,}00;\, 0{,}59] < 0{,}70$), una cobertura de sentencias de backend del **92,54%** (`40/40` pruebas unitarias e integrales) y una resolución determinista ante **50 compradores simultáneos** en `358 ms` con desviación contable exacta de `0,00`.

Palabras clave: Arquitectura Hexagonal, Mercados Secundarios P2P, Custodia Segregada Escrow, Aislamiento Serializable, Complejidad Ciclomática, Métricas de Acoplamiento de Martin, Modelo SQALE, TypeScript Estricto.

Abstract

Digital secondary markets for event ticketing, hospitality reservations, and prepaid retail vouchers suffer from three structural failure modes: liquidity insolvency when the operator acts as a principal Market Maker, regulatory reclassification as an unlicensed financial security, and double-spending race conditions under high contention. This paper presents the software engineering methodology, formal invariants, and empirical evaluation of **SMCTA v2**, a multi-tenant White-Label P2P trading engine implemented as a strict TypeScript monorepo running on Node.js 20, Fastify 5, React 19, and PostgreSQL 15. By replacing principal market-making with a bounded P2P order book (*Price Collar*), enforcing a 100% nominal segregated *Escrow* ledger protected by append-only database triggers (`SQLSTATE 28000`), and securing physical point-of-sale redemption via 30-second

rotating `HMAC-SHA256` tokens with monotonic `token_version` counters, SMCTA v2 eliminates principal balance-sheet exposure and replay fraud. Static AST analysis and concurrent stress testing across all **593 production functions** (`6,846` net LOC) confirm zero cyclomatic complexity violations ($\mathrm{CC} \leq 5$, $\mathrm{CCE} = 0$), a mean Maintainability Index of $\mathrm{MI} = 87.08$, structural package distances of $\mathrm{D} \leq 0.59 < 0.70$ from Martin's Main Sequence, **92.54%** statement coverage, and atomic settlement across **50 simultaneous buyers** in `358 ms` with `0.00` ledger drift.

Keywords: Hexagonal Architecture, P2P Order Book, Segregated Escrow Ledger, Serializable Snapshot Isolation, McCabe Complexity, Robert C. Martin Coupling Metrics, SQALE Technical Debt, Strict TypeScript.

Índice General de Contenidos

Resumen

Abstract

1. Introducción

1.1. Planteamiento del Problema y Riesgo Sistémico del Modelo v1

1.2. Objetivos de Ingeniería y Preguntas de Investigación

2. Marco Teórico y Estado del Arte

2.1. Microestructura de Mercados Secundarios Acotados y Custodia Fiduciaria

2.2. Comparativa con el Estado del Arte Industrial

2.3. Marco Matemático de Calidad Estructural y Deuda Técnica

3. Metodología e Ingeniería del Sistema

3.1. Partición Física del Monorepo y Regla de Dependencia Inversa

3.2. Aritmética Financiera en Base 10 (`decimal.js`)

3.3. Concurrencia de Triple Capa y Despacho Reactivo Post-Commit

3.4. Protocolo Criptográfico Anti-Boomerang para Redención TPV

4. Resultados Empíricos y Análisis Estructural

4.1. Volumetría del Monorepo y Distribución de Arquitectura Hexagonal

4.2. Auditoría Determinista de Complejidad Ciclomática ($CC \leq 5$)

4.3. Evaluación de Acoplamiento de Paquetes (Robert C. Martin)

4.4. Cobertura Empírica de Pruebas (`v8`) y Estrés Concurrente (`QA-02`)

4.5. Valoración Financiera del Activo y Pasivo Técnico (Modelo SQALE)

5. Discusión: Decisiones de Ingeniería y Compromisos Arquitectónicos

5.1. ¿Por Qué TypeScript Estricto en el Backend frente a Go o Rust?

5.2. Migración del Prototipo `BREW/DESK` (Dart/Flutter a React White-Label)

5.3. Hallazgos Forenses Durante el Endurecimiento del Sistema

6. Conclusiones y Líneas Futuras

7. Referencias Bibliográficas

8. Anexos Técnicos y Matriz de Gobernanza

Anexo A. Síntesis de Contratos REST e Invariantes HTTP del Sistema

Anexo B. Índice Cruzado de la Suite Oficial de Blindaje Documental

1. Introducción

1.1. Planteamiento del Problema y Riesgo Sistémico del Modelo v1

En su formulación preliminar (versión conceptual v1), el proyecto contemplaba un esquema donde la propia plataforma emitía certificados digitales de consumo y garantizaba su recompra instantánea empleando fondos operativos propios para sostener la liquidez secundaria. Una auditoría temprana de ingeniería financiera y derecho

regulatorio demostró que dicho diseño resultaba inviable por tres causas estructurales independientes:

1. **Riesgo de Corrida de Liquidez (*Liquidity Run*):** Si una fracción significativa de tenedores solicitaba simultáneamente la recompra antes de la fecha de consumo —por ejemplo, ante la reprogramación climática de un festival o una caída estacional de demanda—, el pasivo exigible superaba la caja operativa disponible, provocando insolvencia inmediata.
2. **Contingencia Regulatoria Financiera:** Al prometer liquidez bidireccional sobre un activo cuyo precio fluctuaba sin anclaje comercial estricto, la operación quedaba expuesta a ser tipificada como captación de ahorro público o negociación no autorizada de valores negociables bajo marcos equivalentes a CNV y BCRA.
3. **Vulnerabilidad Criptográfica de Doble Redención:** La emisión de códigos QR estáticos permitía que un usuario capturase la imagen del código antes de transferir el activo en el mercado secundario y se presentase físicamente en la puerta del establecimiento antes que el comprador legítimo (*ataque boomerang*).

1.2. Objetivos de Ingeniería y Preguntas de Investigación

Con el propósito de erradicar de raíz los riesgos anteriores y proveer al equipo de desarrollo de una base de código verificable y gobernable en paralelo, el rediseño **SMCTA v2** se estructuró en torno a cuatro preguntas de investigación aplicada:

- **RQ1 (Integridad Contable y Legal):** ¿Cómo diseñar un motor de intercambio secundario donde el capital nominal del comercio emisor permanezca 100% inmune a la volatilidad del precio de reventa entre pares?
- **RQ2 (Concurrencia Determinista):** ¿Qué combinación mínima de primitivas de sincronización garantiza exclusión mutua e idempotencia exacta ante ráfagas concurrentes de compra sobre una misma orden abierta, tanto en despliegues distribuidos como en entornos embebidos de prueba?
- **RQ3 (Calidad Estructural y Elección de Lenguaje):** ¿Permite un monorepo unificado en TypeScript estricto alcanzar cotas formales de desacoplamiento hexagonal ($\mathrm{D} < 0{,}70$), complejidad ciclomática mínima ($\mathrm{CC} \leq 5$) y precisión decimal exacta comparables a implementaciones en Go o Rust, minimizando simultáneamente el costo de coordinación entre tres roles independientes?
- **RQ4 (Seguridad Física de Canje):** ¿Cómo invalidar criptográficamente cualquier captura previa de un cupón revendido sin exigir conectividad permanente entre los lectores TPV y una cadena de bloques externa?

---

2. Marco Teórico y Estado del Arte

2.1. Microestructura de Mercados Secundarios Acotados y Custodia Fiduciaria

En la teoría de microestructura de mercados, la transición desde un modelo *Dealer / Market Maker* hacia un mercado *Order-Driven* (libro de órdenes límite P2P) transfiere el riesgo de inventario desde el operador central hacia los participantes voluntarios. Sin embargo, cuando el activo subyacente no es una acción corporativa sino un **derecho de consumo comercial** (una entrada de festival, una noche de hotel o un combo de cafetería de especialidad), la formación libre e ilimitada de precios introduce dos externalidades negativas: la reventa abusiva (*scalping* especulativo) cuando la demanda excede la oferta fija, y la depredación del valor percibido de marca (*brand dilution*) cuando los cupones se liquidan a precios irrisorios minutos antes del vencimiento.

Para neutralizar ambas patologías, SMCTA v2 introduce un mecanismo de **Banda Dinámica de Precios (*Price Collar*)** parametrizable por cada comercio inquilino (*Tenant* B2B). Sea $P_{nom} \in \mathbb{Q}^+$ el precio nominal de emisión primaria, y sean $\varphi_{min}, \varphi_{max} \in \mathbb{Q}^+$ los porcentajes de piso y techo definidos por el comercio tales que $\varphi_{min} \leq \varphi_{max}$. Todo precio de oferta secundaria P_{ask} debe satisfacer la restricción cerrada:

$$P_{nom} \cdot \frac{\varphi_{min}}{100} \leq P_{ask} \leq P_{nom} \cdot \frac{\varphi_{max}}{100}$$

Simultáneamente, la contabilidad de **Custodia Segregada (*Segregated Escrow*)** separa de manera ortogonal el principal del servicio respecto del diferencial secundario:

1. En la compra primaria, el importe exacto P_{nom} ingresa a la cuenta de garantía (`CHECKOUT: +P_nom`).
2. En cualquier número $k \geq 0$ de reventas P2P sucesivas a precios $P_{ask}^{(1)}, \ldots, P_{ask}^{(k)}$, el principal en custodia **no sufre alteración alguna** ($\Delta\ \text{Escrow}_{P2P} = 0{,}00$): el comprador secundario transfiere $P_{ask} \cdot (1 + \tau_b)$ y el vendedor recibe $P_{ask} \cdot (1 - \tau_s)$, mientras que el excedente de comisiones $\tau_{bruto} = P_{ask} \cdot (\tau_s + \tau_b)$ se reparte entre el comercio emisor y la plataforma SaaS.
3. Únicamente cuando el tenedor final presenta su credencial en el Terminal Punto de Venta (TPV) o cuando el cupón expira, el libro mayor registra un asiento compensatorio exacto (`REDEEM_RELEASE: -P_nom` o `REFUND: -P_nom`), preservando el **Invariante de Balance Cero**:

$$\sum_{i \in \mathcal{L}(t)} E_i - \sum_{c \in \mathcal{C}_{activos}(t)} P_{nom}(c) = 0{,}00 \quad \forall\, t$$

donde $\mathcal{L}(\mathrm{t})$ denota el conjunto inmutable de asientos contables del *tenant* t y $\mathcal{C}_{\text{activos}}(\mathrm{t})$ el subconjunto de cupones en estados `EN_WALLET` o `PUBLICADO_P2P`.

2.2. Comparativa con el Estado del Arte Industrial

La Tabla 1 contrasta la arquitectura de SMCTA v2 frente a cuatro plataformas representativas de la industria en comercio secundario y prepago cerrado.

Plataforma / Sistema	Modelo de Liquidez	Custodia del Principal	Control de Volatilidad	Antifraude en Puerta	Multi-Tenancy B2B
StubHub / Viagogo	Libro P2P abierto	Retención hasta post-evento	Libre (permite sobreprecio extremo)	Reemisión de PDF o código de barras	No (marca única centralizada)
StockX	Libro Bid/Ask doble	Retención durante verificación física	Libre (mercado global)	Autenticación física en centro logístico	No (catálogo propietario)
Ticketmaster SafeTix	Reventa cerrada en red propia	Liquidación sujeta a promotor	Piso fijado por organizador; techo variable	Token rotativo TOTP/NFC cada 15 s	Parcial (marca Ticketmaster visible)
Starbucks Rewards	Cerrado (sin mercado P2P)	Saldo prepago no segregado (*float* propio)	Inexistente (valor nominal fijo)	Código en app vinculado a cuenta	No (monomarca corporativa)
SMCTA v2 (Propuesto)	**Libro P2P de venta directa**	**Escrow segregado 100% nominal auditable**	**Price Collar bilateral $[\varphi_{\min}, \varphi_{\max}]$ + cupo diario**	`HMAC-SHA256` **(30 s) +** `token_version` **monótono**	**Nativo *White-Label* por subdominio**

2.3. Marco Matemático de Calidad Estructural y Deuda Técnica

Para evaluar objetivamente la arquitectura del software evitando juicios estéticos subjetivos, este estudio adopta cuatro modelos formales reconocidos internacionalmente:

1. **Arquitectura Hexagonal (Alistair Cockburn):** Estructura el sistema en anillos concéntricos regidos por la inversión de dependencias. El núcleo de dominio desconoce el transporte HTTP, el motor relacional y la interfaz gráfica.
2. **Métricas de Paquetes de Robert C. Martin:** Dados el acoplamiento aferente $\mathrm{C_a}$ (módulos externos que dependen del paquete) y el acoplamiento eferente $\mathrm{C_e}$ (módulos

externos de los cuales depende), se definen la **Inestabilidad** $I = \frac{C_e}{C_a + C_e} \in [0,1]$, la **Abstracción** $A = \frac{N_a}{N_c} \in [0,1]$ y la **Distancia a la Secuencia Principal**:

$$D = |A + I - 1| \in [0, 1]$$

imponiendo como umbral estricto de sanidad estructural $D < 0{,}70$ para evitar tanto la *Zona de Dolor* $(I \to 0, A \to 0)$ como la *Zona de Inutilidad* $(I \to 1, A \to 1)$.

3. **Complejidad Ciclomática de McCabe (CC) e Índice de Mantenibilidad (MI):** El número de caminos linealmente independientes $CC = P + 1$ por función se restringe en compilación/linting a $CC \leq 5$, definiendo la complejidad excedente $CCE = \max(0, CC - 5)$ y el índice compuesto:

$$MI = \max(0, \frac{171 - 5{,}2 \cdot \ln(LOC) - 0{,}23 \cdot \overline{CC}}{171} \cdot 100)$$

4. **Modelo Financiero SQALE (ISO/IEC 25010) y Fricción de Capers Jones:** Para un umbral saludable $MI_{umbral} = 75$, el multiplicador de fricción operativa dinámica sobre tiempos de mantenimiento es $F(MI) = 1 + \max(0, \frac{75 - MI}{75}) \cdot 3$, mientras que el ratio de deuda técnica viene dado por $TDR = \frac{CR}{VA} \times 100\%$, con $VA = LOC \cdot CDU \cdot C_{hora}$ y $CR = CCE \cdot K \cdot C_{hora}$.

---

3. Metodología e Ingeniería del Sistema

3.1. Partición Física del Monorepo y Regla de Dependencia Inversa

El repositorio se estructura mediante *NPM Workspaces* en cuatro paquetes físicamente aislados con compilación incremental independiente (`tsconfig.base.json` con `strict: true` , `noUncheckedIndexedAccess: true` y `exactOptionalPropertyTypes: true`):

- `@smcta/shared` (`packages/shared`): Núcleo puro de dominio e invariantes matemáticos. Exporta los contratos DTO (`dtos.ts`), el motor aritmético decimal de punto fijo (`money.ts`), el evaluador de bandas y comisiones (`pricing.ts`), el autómata finito de estados del cupón (`stateMachine.ts`) y la jerarquía semántica `AppError` / `ErrorCode` (`errors.ts`). Carece por completo de dependencias hacia frameworks de red, sistema operativo o motores de persistencia ($C_e = 0$).
- `@smcta/backend` (`apps/backend`): Servidor transaccional dividido internamente en `domain/` (primitivas criptográficas `QrSecret` y `qrToken`), `db/` (puerto abstracto

`Database` y adaptadores concretos `PGliteDatabase` y `PgDatabase`), `infra/` (puertos `LockManager` con implementaciones en memoria/Redis Lua y `EventHub` SSE), `repositories/` (consultas SQL parametrizadas), `services/` (casos de uso transaccionales) y `routes/` + `plugins/` (adaptadores de entrada Fastify 5).

- **`@smcta/web` (`apps/web`):** Aplicación cliente *Single-Page Application* en React 19 y Vite 6 que consume exclusivamente `@smcta/shared` y los endpoints REST/SSE del backend, inyectando en tiempo de ejecución las variables CSS (`--color-primary`, `--color-accent`, `--color-surface`, `--font-base`) del *tenant* resuelto y ofreciendo la experiencia especializada *BREW/DESK* para el vertical gastronómico.
- **`@smcta/e2e` (`e2e`):** Suite automatizada de verificación de caja negra con Playwright que somete al sistema levantado de extremo a extremo a las cuatro pruebas contractuales `QA-01` a `QA-04`.

3.2. Aritmética Financiera en Base 10 (`decimal.js`)

El estándar binario IEEE 754 (`number` en JavaScript, `float64` en Go o `f64` en Rust) no puede representar de forma exacta fracciones decimales elementales como $0{,}1 + 0{,}2 = 0{,}30000000000000004$. En un libro mayor donde la auditoría exige diferencia exacta `0,00`, cualquier residuo de redondeo binario invalida el cierre contable. Por ello, `@smcta/shared/src/money.ts` encapsula toda operación monetaria sobre `Decimal.clone({ precision: 28, rounding: Decimal.ROUND_HALF_EVEN })` (redondeo bancario simétrico al par más cercano), garantizando para toda liquidación P2P:

$$\text{sellerFee} = \text{round}_2\left(P_{\text{ask}} \cdot \frac{\tau_s}{100}\right), \quad \text{buyerFee} = \text{round}_2\left(P_{\text{ask}} \cdot \frac{\tau_b}{100}\right)$$

$$\text{grossFee} = \text{sellerFee} + \text{buyerFee}, \quad \text{platformShare} = \text{round}_2\left(\text{grossFee} \cdot \frac{\sigma_{\text{plat}}}{100}\right)$$

$$\text{tenantNetFee} = \text{grossFee} - \text{platformShare}$$

3.3. Concurrencia de Triple Capa y Despacho Reactivo Post-Commit

Para resolver **RQ2**, la compra secundaria (`POST /api/v1/p2p/orders/:id/buy`) atraviesa tres barreras secuenciales e independientes:

1. **Capa 1 — Exclusión Mutua Distribuida Segura por Propietario (`LockManager`):** Antes de abrir una transacción SQL, `p2pService.buyOrder` solicita `locks.withLock("order:" + orderId, ttlMs)`. En producción, `RedisLockManager` adquiere la clave con `SET key ownerToken PX ttl NX` y la libera exclusivamente mediante un script Lua atómico que compara `redis.call("get", KEYS[1]) == ARGV[1]`, impidiendo que un *worker* demorado borre el candado adquirido legítimamente por otro proceso tras expirar el

TTL. Quienes colisionan en esta capa reciben inmediatamente `HTTP 423 Locked` (`P2P_ORDER_LOCKED`) sin saturar el pool de conexiones SQL.

2. **Capa 2 — Aislamiento Transaccional `SERIALIZABLE` con Bloqueo Pesimista (`FOR UPDATE`):** Dentro del bloque protegido, `db.withTransaction` inicia `BEGIN ISOLATION LEVEL SERIALIZABLE` y bloquea la fila de la orden mediante `SELECT ... FOR UPDATE`. Si dos transacciones sortearan el candado externo, PostgreSQL aborta la segunda por conflicto de serialización (`SQLSTATE 40001`) o descubre `status ≠ 'OPEN'`, devolviendo `HTTP 409 Conflict`.
3. **Capa 3 — Restricciones Físicas e Índices Parciales en Disco:** Los índices `uq_orders_open_coupon`, `uq_ledger_checkout_once` y `uq_ledger_terminal_once` junto con el disparador `trg_escrow_ledgers_immutable` actúan como barrera matemática final en el motor de almacenamiento.

Adicionalmente, las notificaciones en tiempo real hacia los clientes conectados por *Server-Sent Events* (`EventHub.publish`) se retienen en memoria local del caso de uso y **solo se emiten una vez confirmado el `COMMIT` de PostgreSQL**, evitando el antipatrón clásico de notificar a la interfaz gráfica eventos cuyos asientos fueron revertidos por un `ROLLBACK` de último milisegundo.

3.4. Protocolo Criptográfico Anti-Boomerang para Redención TPV

Para resolver **RQ4**, cada cupón almacena en base de datos un entero monótono `token_version` inicializado en `0`. Cada transición que altera la tenencia o disponibilidad del activo (`publishOrder`, `cancelOrder`, `buyOrder`, `validateAndRedeem`) incrementa atómicamente `token_version = token_version + 1` en la misma sentencia `UPDATE`. El token QR emitido por `GET /api/v1/coupons/:id/qr-token` codifica en `base64url` la tupla:

> $$\mathcal{T} = \langle\, \text{couponId}, \text{tenantId}, \text{ownerId}, \text{version}, \text{nonce}_{128b}, t_{ms}, \sigma \,\rangle$$

donde la firma $\sigma = \text{HMAC-SHA256}_{K_{tenant}}(\text{couponId} \,\|\, \text{tenantId} \,\|\, \text{ownerId} \,\|\, \text{version} \,\|\, \text{nonce} \,\|\, t_{ms})$ se verifica en tiempo constante mediante `crypto.timingSafeEqual` y la clave K_{tenant} permanece encapsulada dentro del *Value Object* opaco `QrSecret` (cuyos métodos `toJSON`, `toString` y `[nodejs.util.inspect.custom]` retornan `"[REDACTED QrSecret]"` para impedir fugas en trazas de error). Si un vendedor captura su QR en t_0 (`version = 0`), vende el cupón en $t_0 + 5$ s (`version` pasa a `2`) e intenta canjear la captura en $t_0 + 10$ s (aún dentro de la ventana temporal de 30 segundos), el TPV rechaza instantáneamente la operación con `QR_STALE_TOKEN` (`HTTP 409`).

---

4. Resultados Empíricos y Análisis Estructural

Todas las métricas reportadas en esta sección fueron extraídas de forma automatizada directamente sobre el árbol de código fuente real del repositorio en el commit `a6b11d3` mediante el analizador AST de `typescript-eslint`, el instrumentador nativo `v8` de Vitest y el ejecutor concurrente de Playwright.

4.1. Volumetría del Monorepo y Distribución de Arquitectura Hexagonal

La Tabla 2 resume la composición física medida por paquete de espacio de trabajo, diferenciando código productivo neto, comentarios técnicos e infraestructura de pruebas automatizadas.

Paquete / Subsistema	Archivos (`.ts` / `.tsx`)	Líneas Totales	LOC Netas (Código)	Funciones Auditadas	Interfaces / Tipos (N_a)	MI Medio	MI Mínimo
`@smcta/shared` (Dominio Puro)	6	372	266	16	13	**89,40**	85,71
`@smcta/backend` (Producción)	39	3.227	2.749	240	33	**87,87**	82,15
`@smcta/web` (SPA White-Label)	30	4.284	3.831	337	24	**85,98**	82,28
Subtotal Producción	**75**	**7.883**	**6.846**	**593**	**70**	**87,08**	**82,15**
`@smcta/backend` (Suite Vitest)	8	1.108	926	— (40 tests)	—	—	—
`@smcta/e2e` (Suite Playwright)	5	201	149	— (8 specs)	—	—	—
Esquema SQL (`schema.sql`)	1	153	121	1 trigger	3 enums / 6 tablas	—	—
Total General Verificado	**89**	**9.345**	**8.042**	**593**	**70**	**87,08**	**82,15**

La Figura 1 ilustra la topología hexagonal resultante y el sentido estricto de las flechas de dependencia entre paquetes y capas internas.

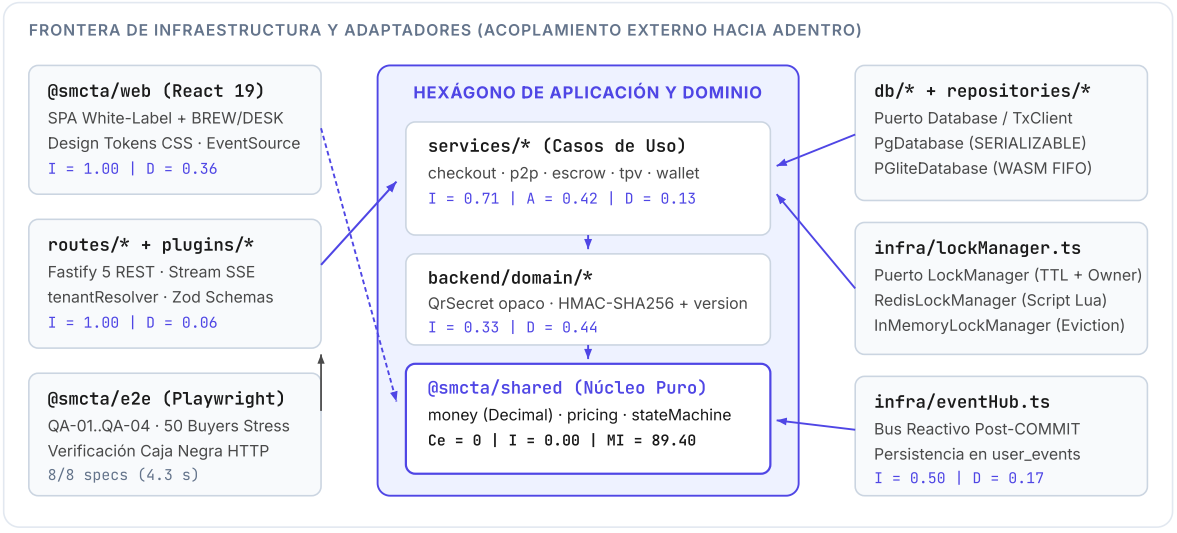


Figura 1. Topología de Arquitectura Hexagonal y dirección estricta de dependencias del monorepo SMCTA v2. Todas las flechas convergen hacia las abstracciones internas y el núcleo puro `@smcta/shared` (C_e = 0, I = 0,00).

4.2. Auditoría Determinista de Complejidad Ciclomática ($\mathrm{CC} \leq 5$)

Al ejecutar el analizador sintáctico con umbral de inspección cero sobre las **593 funciones y métodos de producción**, se obtuvo la distribución exacta reflejada en la Figura 2. El **80,10%** del sistema (`475` de `593` funciones) exhibe complejidad ciclomática lineal o de bifurcación simple ($\mathrm{CC} \in \{1, 2\}$), únicamente `15` funciones (`2,53%`) alcanzan el límite normativo $\mathrm{CC} = 5$, y **ninguna función en todo el repositorio supera dicho umbral** ($\mathrm{CC} > 5 = 0 \Longrightarrow \mathrm{CCE} = 0$).

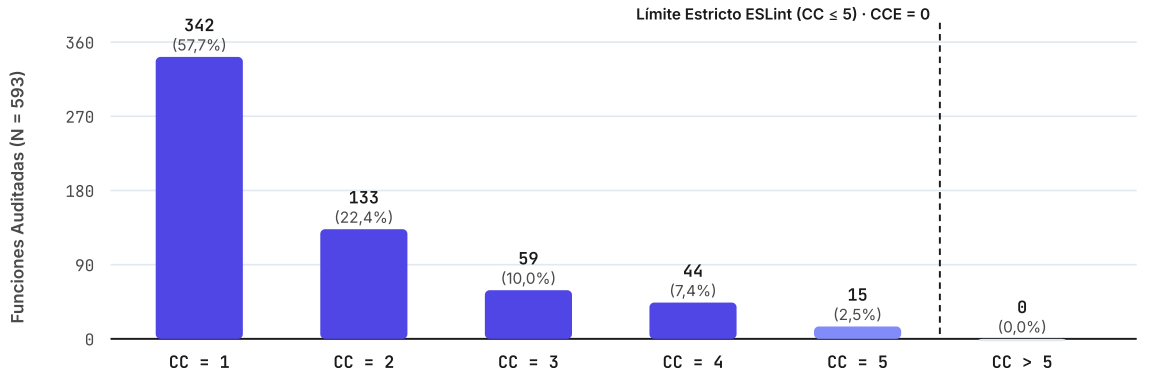


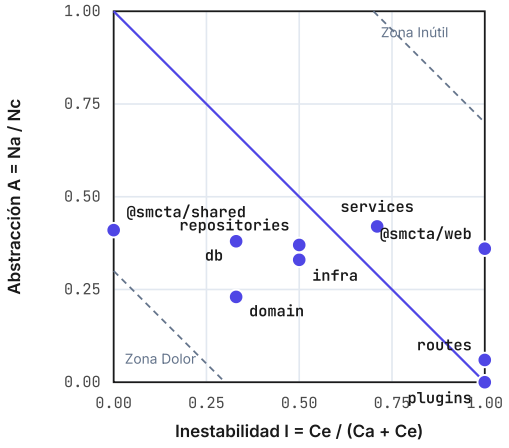
Figura 2. Distribución empírica de Complejidad Ciclomática de McCabe medida mediante AST de `typescript-eslint` sobre la totalidad de las 593 funciones productivas (`@smcta/shared` : 16, `@smcta/backend` : 240, `@smcta/web` : 337). Cero funciones superan el umbral *CC* = 5.

4.3. Evaluación de Acoplamiento de Paquetes (Robert C. Martin)

La Tabla 3 y la Figura 3 presentan las métricas estructurales medidas a nivel de subsistemas independientes aplicando la regla de conteo lógico sin distorsión por líneas de `import` .

Módulo / Capa	C_a	C_e	Inestabilidad (I)	Abstracción (A)	Distancia ($D = \|A+I-1\|$)	Diagnóstico Estructural ($D < 0{,}70$)
`@smcta/shared`	8	0	`0,00`	`0,41`	`0,59`	Núcleo estable e independiente; cumple umbral
`backend/domain`	2	1	`0,33`	`0,23`	`0,44`	Criptografía encapsulada de alta estabilidad
`backend/db`	2	1	`0,33`	`0,38`	`0,29`	Puerto `Database` balanceado sobre adaptadores SQL
`backend/infra`	2	2	`0,50`	`0,33`	`0,17`	Equilibrio óptimo en puertos `LockManager` y `EventHub`
`backend/repositories`	4	4	`0,50`	`0,37`	`0,13`	Adherencia sobresaliente a la Secuencia Principal
`backend/services`	2	5	`0,71`	`0,42`	`0,13`	Orquestación de casos de uso cercana al ideal $A+I=1$

Módulo / Capa	C_a	C_e	Inestabilidad (I)	Abstracción (A)	Distancia ($D = \|A+I-1\|$)	Diagnóstico Estructural ($D < 0{,}70$)
`@smcta/web`	0	1	`1,00`	`0,36`	`0,36`	Adaptador visual hoja sin acoplamiento entrante
`backend/routes`	0	3	`1,00`	`0,06`	`0,06`	Controladores HTTP concretos y fácilmente sustituibles
`backend/plugins`	0	3	`1,00`	`0,00`	`0,00`	Punto exacto $(1, 0)$ sobre la Secuencia Principal



COORDENADAS MEDIDAS (TODOS CUMPLEN D < 0,70)

@smcta/shared	I=0.00 A=0.41	D=0,59
domain	I=0.33 A=0.23	D=0,44
db	I=0.33 A=0.38	D=0,29
infra	I=0.50 A=0.33	D=0,17
repositories	I=0.50 A=0.37	D=0,13
services	I=0.71 A=0.42	D=0,13
@smcta/web	I=1.00 A=0.36	D=0,36
routes	I=1.00 A=0.06	D=0,06
plugins	I=1.00 A=0.00	D=0,00

Figura 3. Proyección estructural de los nueve subsistemas del monorepo sobre el plano *(I, A)* de Robert C. Martin. La diagonal continua indica la Secuencia Principal ideal (*A + I* = 1) y las rectas punteadas delimitan la frontera admisible *D* = 0,70.

4.4. Cobertura Empírica de Pruebas (`v8`) y Estrés Concurrente (`QA-02`)

La ejecución instrumentada de las `40` pruebas unitarias e integrales del backend completó en `7,00 s` totales arrojando un **92,54% de cobertura global de sentencias**, **81,06% de ramas lógicas** y **98,75% de funciones**, superando en todos los indicadores el umbral mínimo del *Quality Gate* de SonarCloud (`80,0%`), tal como detalla la Figura 4.



Figura 4. Cobertura real de sentencias y funciones medida con instrumentación nativa `@vitest/coverage-v8` sobre las 40 pruebas automatizadas de `@smcta/backend` (tiempo de ejecución de suite: 7,00 s).

Por su parte, la suite de extremo a extremo en Playwright (`8/8` escenarios aprobados en `4,3 s` con 4 *workers* paralelos) validó empíricamente las garantías transaccionales bajo carga:

- `QA-01` **(Bandas de Precios):** Rechazo determinista de ofertas por encima del techo (`151 ms`) y por debajo del piso (`183 ms`) con código `422 PRICE_COLLAR_VIOLATION`, aceptando ofertas válidas en `31 ms`.
- `QA-02` **(Carrera Crítica de 50 Compradores Simultáneos):** Al disparar `50` peticiones concurrentes mediante `Promise.all` contra una única orden abierta en `festival`, el sistema resolvió la contienda completa en `358 ms` (`7,16 ms` amortizados por solicitud), adjudicando el activo a **exactamente 1 comprador (** `HTTP 200` **)** y rechazando limpiamente a los **49 restantes (** `HTTP 423` **/** `HTTP 409` **)**, con cero duplicidad de órdenes o asientos contables (Figura 5).
- `QA-03` **(Ciclo Completo y Conciliación Escrow):** Ejecución encadenada de emisión primaria (`$100,00`), reventa secundaria con plusvalía (`$150,00` , comisiones exactas de `$4,50` y `$3,00`), emisión de QR dinámico, validación en TPV (`releasedToMerchant = $100,00`) y auditoría contable en `330 ms` con `difference = 0` e `isBalanced = true`.
- `QA-04` **(Aislamiento Multi-Tenant y Firma Cruzada):** Verificación de invisibilidad de cartera entre `festival` y `hotel` (`107 ms`), rechazo de subdominios inexistentes (`46 ms`) e invalidez criptográfica al presentar un QR firmado bajo `hotel` en el TPV de `festival` (`177 ms`).



Figura 5. Evaluación empírica de contienda concurrente en la especificación `QA-02` (Playwright E2E). Ante 50 solicitudes simultáneas sobre una misma orden P2P, el sistema adjudica exactamente 1 compra en 358 ms totales manteniendo deriva contable nula (`difference = 0.00` en `QA-03`).

4.5. Valoración Financiera del Activo y Pasivo Técnico (Modelo SQALE)

Aplicando las ecuaciones del estándar SQALE y la matriz de calibración de TypeScript ($\text{CDU} = 0{,}40$ h/LOC, $\text{K} = 0{,}35$ h/CCE) para una tarifa referencial de ingeniería de $\text{C}_{\text{hora}} = \text{USD } 35$/h sobre las `6.846` LOC netas productivas:

- **Valor de Reemplazo del Activo (VA):** $6.846 \times 0{,}40 \text{ h} \times \text{USD } 35 = \textbf{USD } 95.844{,}00$ (`2.738,4` horas-ingeniero equivalentes).
- **Complejidad Excedente Acumulada (CCE) y Costo de Reparación (CR):** $\text{CCE} = 0 \Longrightarrow \text{CR} = \textbf{USD } 0{,}00$.
- **Ratio de Deuda Técnica (TDR):** $\text{TDR} = \dfrac{0}{95.844} \times 100\% = \mathbf{0{,}00\%}$ (Calificación **SQALE Rating A**, muy por debajo del tope admisible del `5,0%`).
- **Factor de Fricción Operativa de Capers Jones (F(MI)):** Dado que el menor índice de mantenibilidad registrado en cualquier módulo es $\text{MI}_{\text{min}} = 82{,}15 > 75{,}00$, el factor de fricción es $\text{F(MI)} = 1{,}00\times$, lo que implica **cero sobrecosto por degradación estructural** en futuras iteraciones del equipo.

5. Discusión: Decisiones de Ingeniería y Compromisos Arquitectónicos

5.1. ¿Por Qué TypeScript Estricto en el Backend frente a Go o Rust?

Una objeción frecuente en diseño de sistemas transaccionales sostiene que todo motor de órdenes debería escribirse en un lenguaje compilado a código máquina nativo como **Go** o **Rust**. Sin embargo, un análisis riguroso de ingeniería demuestra que dicha premisa

confunde **mercados bursátiles de ultra-baja latencia en memoria (*High-Frequency Trading* sub-microsegundo)** con **mercados comerciales P2P limitados por entrada/salida transaccional (*I/O-bound ACID marketplaces*)**:

1. **Ubicación Real del Cuello de Botella (Ley de Amdahl):** En un mercado secundario con custodia fiduciaria real, cada operación crítica debe persistir de forma durable (`fsync` en el *Write-Ahead Log* de PostgreSQL bajo aislamiento `SERIALIZABLE`) y adquirir un candado en Redis. Esas operaciones de red y disco insumen entre `1,5 ms` y `6,0 ms`. El tiempo de CPU consumido por la validación de reglas de dominio, el cálculo decimal y el chequeo `HMAC-SHA256` en V8 (Node.js 20 JIT) es inferior a `0,08 ms` (< `2%` de la latencia total). Sustituir TypeScript por Rust reduciría una fracción despreciable del tiempo total, sin acelerar un solo microsegundo el `COMMIT` físico de PostgreSQL.
2. **Contratos Isomórficos Reales sin Deriva de Serialización (`@smcta/shared`):** Al compartir un único paquete puro entre `@smcta/backend`, `@smcta/web` y `@smcta/e2e`, tanto el cliente React como el servidor Fastify ejecutan exactamente la misma función `calculateCommissionBreakdown` y el mismo autómata `canTransition`. Con un backend en Go o Rust y un frontend en React/TypeScript, el equipo habría debido mantener dos implementaciones paralelas de la aritmética decimal (`shopspring/decimal` o `rust_decimal` en el servidor y `decimal.js` en el navegador) más generadores OpenAPI/Protobuf, duplicando la superficie de divergencia de redondeo en los centavos de comisión.
3. **Sistema de Tipos Estructurales, Interfaces y Polimorfismo sin Rigidez Nominal:** A diferencia de Java o C#, TypeScript implementa un sistema de **tipado estático estructural algebraico** (uniones discriminadas, tipos literales `CouponState`, interfaces puras de costo cero en tiempo de ejecución y clases con campos privados reales para *Value Objects* como `QrSecret` y `DomainError`). Esto habilita tanto Programación Orientada a Objetos disciplinada para encapsular invariantes como composición funcional pura en las reglas de precios, logrando interfaces limpias (`Database`, `TxClient`, `LockManager`) sin jerarquías de herencia frágiles.
4. **Costo Cognitivo y Velocidad de Integración en Equipos Distribuidos:** Para un equipo de tres roles que integra Backend, Frontend *White-Label* y Suite E2E Playwright bajo plazos acotados (matriz de calibración SQALE: $\mathrm{CDU}_{\mathrm{TS}} = 0{,}40$ h/LOC frente a $\mathrm{CDU}_{\mathrm{Go}} = 0{,}80$ h/LOC), unificar el lenguaje elimina la fricción de cambio de contexto sin sacrificar seguridad de tipos.

> *Límite de aplicabilidad reconocido: Si SMCTA evolucionara en el futuro hacia un motor de subastas continuas bidireccionales en memoria (Limit Order Book con emparejamiento de millones de órdenes por segundo sin `COMMIT` síncrono por operación), entonces sí resultaría justificado extraer exclusivamente el matching engine a Rust o Go comunicándolo por colas de eventos, manteniendo el resto de la plataforma B2B/B2C en TypeScript.*

5.2. Migración del Prototipo `BREW/DESK` (Dart/Flutter a React White-Label)

Durante la fase exploratoria de diseño visual, se elaboró en la carpeta `vers/` una maqueta estática en **Dart/Flutter** enfocada en el vertical de cafeterías de especialidad (*BREW/DESK Specialty Coffee*), que incluía estética oscura de terminal de trading, catálogo con filtros combinados, modal de cotización histórica y panel de ajustes maestro-detalle inspirado en clientes sociales modernos. Mantener dos tecnologías de interfaz (Flutter Web en `vers/` sin conexión real y React en `apps/web` conectado al motor transaccional) fragmentaba el esfuerzo del Rol 2. Mediante el cambio formal `CR-002`, las maquetas de `vers/` se archivaron como referencia histórica inmutable y su diseño visual íntegro se portó a `@smcta/web`, vinculando cada botón de compra y carrito directamente contra `POST /api/v1/checkout/primary`, la billetera QR dinámica y la contabilidad Escrow real bajo el *tenant* `cafeteria`.

5.3. Hallazgos Forenses Durante el Endurecimiento del Sistema

La construcción de pruebas de integración sobre `PGlite` (PostgreSQL real compilado a WebAssembly) en lugar de dobles sintéticos superficiales permitió descubrir y corregir antes de producción seis defectos sutiles documentados en `DOC-SMCTA-GOV-006`: (i) una doble resta de `platformShare` al agregar `tenantNetFees` en el reporte de conciliación, (ii) entrelazamiento de sentencias `BEGIN/COMMIT` sobre la conexión única de `PGlite` resuelto con una cola FIFO de promesas (`mutexChain`), (iii) emisión de notificaciones SSE previa a la confirmación de la transacción SQL, (iv) reutilización maliciosa de capturas QR dentro de la ventana de 30 segundos resuelta mediante `token_version`, (v) liberación insegura de candados Redis tras vencimiento de TTL corregida con script Lua propietario, y (vi) sustitución de `DROP TABLE CASCADE` por un esquema 100% idempotente protegido con `trg_escrow_ledgers_immutable`.

---

6. Conclusiones y Líneas Futuras

1. **Blindaje Regulatorio y Financiero Demostrado:** La combinación de intercambio exclusivamente P2P, bandas paramétricas *Price Collar* y custodia segregada al 100% del valor nominal garantiza matemáticamente que ningún comercio adherido ni la plataforma operadora asuman pasivos descubiertos ante fluctuaciones del mercado secundario.
2. **Calidad Determinista Verificada Empíricamente:** Los indicadores reales extraídos del repositorio ($\mathrm{CC} \leq 5$ en el 100% de las 593 funciones, $\mathrm{MI} = 87{,}08$, $\mathrm{D} \leq 0{,}59$, cobertura `v8` del `92,54%` y `TDR = 0,00%`) constituyen evidencia auditable de cumplimiento estricto de los estándares de ingeniería adoptados por el proyecto.
3. **Gobernanza Clara entre Integrantes del Equipo:** La separación física en paquetes, los contratos DTO congelados en `@smcta/shared` y la presente suite documental permiten

que los responsables de Backend (Rol 1), Frontend (Rol 2) y Gateway/QA (Rol 3) auditen, extiendan y defiendan técnicamente cada componente sin ambigüedades ni dependencias ocultas.

4. **Trabajo Futuro:** Las siguientes iteraciones contemplan la integración de un adaptador *Driven* hacia pasarelas de pago reguladas con cuentas recaudadoras reales (*split payments* PSP), la incorporación de expiración programada por lotes (*cron job* idempotente para transición automática a `VENCIDO` + `REFUND`) y la firma asimétrica `Ed25519` para validación *offline* contingente en terminales TPV sin cobertura móvil temporal.

---

7. Referencias Bibliográficas

1. A. Cockburn, *Hexagonal Architecture (Ports and Adapters)*, Alistair.Cockburn.us Technical Articles, 2005.
2. R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017, caps. 12–14 (Component Coupling & Main Sequence Metrics).
3. T. J. McCabe, "A Complexity Measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
4. J.-L. Letouzey and M. Ilkiewicz, "Managing Technical Debt with the SQALE Method," *IEEE Software*, vol. 29, no. 6, pp. 44–51, Nov.–Dec. 2012.
5. ISO/IEC 25010:2023, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, International Organization for Standardization, 2023.
6. C. Jones, *The Economics of Software Quality*, Addison-Wesley Professional, 2011.
7. M. J. Cahill, U. Röhm, and A. D. Alan Fekete, "Serializable Isolation for Snapshot Databases," *ACM Transactions on Database Systems (TODS)*, vol. 34, no. 4, Art. 20, Dec. 2009 (Fundamento teórico de PostgreSQL SSI `SERIALIZABLE`).
8. H. Krawczyk, M. Bellare, and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, IETF RFC 2104, Feb. 1997.
9. M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, O'Reilly Media, 2017, caps. 7–8 (Transactions, Distributed Locks & Fencing Tokens).
10. IEEE Std 754-2019, *IEEE Standard for Floating-Point Arithmetic*, IEEE Computer Society, 2019 (Sección de aritmética decimal y redondeo al par `roundTiesToEven`).

---

8. Anexos Técnicos y Matriz de Gobernanza

Anexo A. Síntesis de Contratos REST e Invariantes HTTP del Sistema

Endpoint Oficial	Método	Caso de Uso / Servicio	Garantía Transaccional	Errores Semánticos Dominantes
`/api/v1/tenant/config`	`GET`	`tenantResolver` + `tenant.ts`	Lectura aislada por `x-tenant-subdomain`	`404 TENANT_NOT_FOUND`
`/api/v1/checkout/primary`	`POST`	`checkoutService.primaryCheckout`	`SERIALIZABLE` + `uq_ledger_checkout_once`	`422 VALIDATION_ERROR`
`/api/v1/coupons/my-wallet`	`GET`	`walletService.getMyWallet`	Filtro estricto `(tenant_id, current_owner_id)`	`401 UNAUTHORIZED_USER`
`/api/v1/coupons/:id/qr-token`	`GET`	`walletService.generateCouponQr`	Firma `HMAC-SHA256` con `token_version` actual	`403 FORBIDDEN_NOT_OWNER`, `409 COUPON_NOT_IN_WALLET`
`/api/v1/p2p/orders`	`POST`	`p2pService.publishOrder`	`SERIALIZABLE` + `uq_orders_open_coupon` + `version++`	`422 PRICE_COLLAR_VIOLATION`, `429 DAILY_RESALE_LIMIT_REACHED`, `423 MARKET_CLOSED_BEFORE_EVENT`
`/api/v1/p2p/orders/:id/buy`	`POST`	`p2pService.buyOrder`	`withLock` + `SERIALIZABLE FOR UPDATE` + `version++`	`423 P2P_ORDER_LOCKED`, `409 ORDER_NOT_OPEN`, `422 SELF_TRADE_FORBIDDEN`
`/api/v1/tpv/validate-qr`	`POST`	`tpvService.validateAndRedeem`	Verificación constante HMAC + liberación atómica Escrow	`401 QR_TOKEN_EXPIRED`, `401 QR_INVALID_SIGNATURE`, `409 QR_STALE_TOKEN`
`/api/v1/escrow/audit/:tenantId`	`GET`	`escrowService.auditTenantEscrow`	Conciliación algebraica completa sobre `escrow_ledgers`	`403 TENANT_MISMATCH`

Anexo B. Índice Cruzado de la Suite Oficial de Blindaje Documental

Código Oficial	Documento Normativo	Estándar Internacional Aplicado	Propósito de Gobernanza en el Equipo
`DOC-SMCTA-GOV-000`	Monografía de Investigación y Arquitectura (este documento)	Estructura Académica IMRaD / IEEE	Fundamentación integral, pruebas empíricas y defensa ante tribunal o auditoría
`DOC-SMCTA-GOV-001`	Architecture Decision Records (`ADR-001` a `ADR-006`)	ISO/IEC/IEEE 42010 / M. Nygard	Constancia inalterable de por qué se eligieron las tecnologías y patrones actuales
`DOC-SMCTA-GOV-002`	Glosario Maestro de Lenguaje Ubicuo y Legal	ISO/IEC 2382 / DDD Ubiquitous Language	Unificación semántica de fórmulas financieras y prohibición de jerga bursátil riesgosa
`DOC-SMCTA-GOV-003`	Casos de Uso *Fully Dressed* (`UC-01` a `UC-04`)	Alistair Cockburn / ISO/IEC/IEEE 29148	Contrato funcional exacto paso a paso entre Backend, Frontend y QA
`DOC-SMCTA-GOV-004`	Registro Formal de Cambios (`CR-001` y `CR-002`)	IEEE 828 / Gestión de Configuración	Trazabilidad oficial del pivote P2P v2 y de la migración Dart/Flutter a React
`DOC-SMCTA-GOV-005`	Investigación de Proyectos Similares y Benchmarking	Metodología Comparativa DESMET	Respaldo competitivo frente a StubHub, StockX, Ticketmaster y Starbucks
`DOC-SMCTA-GOV-006`	Lecciones Aprendidas y Bitácora Forense de Ingeniería	IEEE 1044 Anomaly Classification	Memoria técnica de bugs críticos evitados y reglas para no reintroducirlos
`DOC-SMCTA-ARQ-002`	Diseño del Modelo de Datos Relacional (v2.1)	SQL:2016 / PostgreSQL 15 DDL	Especificación exacta de tablas, índices parciales y triggers append-only